



An introduction to MODEL VIEW CONTROLLER

Kshitij Sobti

If you've ever trawled the internet looking for information on programming in your favourite language, you must have come across the term model view controller (MVC). Ever wondered what it was, and why people make such a fuss about it? Well, here's your chance to find out.

It's a principle that has over time been proven to assuage several issues that come up while developing large software projects. It isn't limited to any specific programming language, instead it is a paradigm for organising the code and functionality of your application in such a way that it is easily extendible and easier to maintain.

Design Patterns

You may have heard a lot of people talk about how it is best to separate the presentation from the content. Most programs these days try to do just that. The reason this is important is, that more often than not, it is only one of them that needs updating.

Imagine you write a long and complicated report with a proper structure such as title, headings, sub-headings, illustrations, comments and so on – each with a different formatting. Imagine then if you're told it has to be submitted in different standardised formatting! It would be really troublesome to go and change each and every thing in your entire document!

The solution for this is to define a particular style and its function, rather than apply styles to the content. Let's compare the following two sentences:

- The font size 22 in my article is 'The Early Man'. After that, I begin with an 18 point text 'Introduction', followed by a 14-point text "OR"
- The title of my article is 'The Early Man', after that I begin with my first chapter, titled 'Introduction' followed by the content.

Similarly, a computer can work better with content if it knows the intended purpose. For example, in Microsoft Word if you define text as the title, some as the subtitle, and some as the content, then all you need to do to change the appearance of the entire document, is to change the styles associated with those types of data. In addition to this, Word can easily generate a table of contents for you if you've gone through the effort of defining headings and sub-headings.

In other words, we try to divide the content into two parts - one which contains the data and its description. Additionally, there is information on how to display this data. Let us say we want text as the accompanying box.

Earlier, this is how the text might be represented:

Font: Amplitude, Size: 18:

The Early Man

Font: Amplitude, Size: 14:

Introduction

Font: Calibri, Size: 11:

The early man was the earliest man to ever be called a man.

However, the following method is now used to present the information:

Heading: The Early Man

Sub-Heading:

Introduction

Content: The early man was the earliest man to ever be called a man.

Along with: Heading = Font: Cambria, Size: 18

Sub-Heading : Font: Cambria, Size: 14

Content : Font: Calibri, Size: 11

The advantages of the second approach are quite evident.

The Early Man Introduction

The early man was the earliest man to ever be called a man.

Now it would be possible for you to change the font of all headings, by merely changing one line.

While the second approach may seem to be taking more space, this is not entirely true. As the content increases, the amount of space taken up by formatting information in case of separate presentation and content can be much less.

The MVC Paradigm

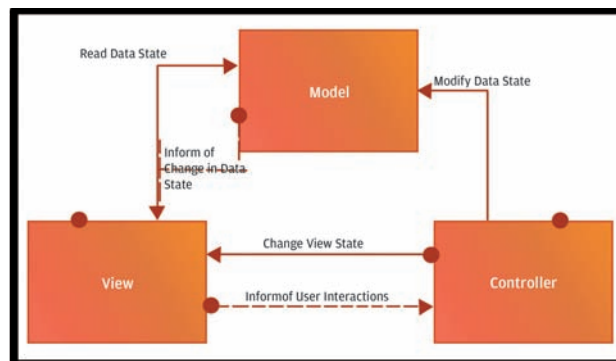
So what does this have to do with programming? For one, most programs are centred on data and display. A majority of the software we use accesses some kind of data and displays something on screen. The data need not be something we would concern ourselves with, but it is there.

To use the MVC paradigm, we separate the functionality of our application into three parts, the model, the view and the controller. The model is the data store of the application, the view / views are what decide how data is displayed, and the controller is what handles interaction and response. It is not as simple a job as it sounds, it can often be quite counter-intuitive, and sometimes might not even be the best thing to do.

The model

The model is the repository of data for the entire application. Any information that the program works with is part of the model. By decoupling this from the other parts of the program we ensure that the program will continue working as expected even if we decide to change the way we store or access data.

For example, if you develop a simple application for handling contacts, and address book. You initially start off using text files while developing the application. However, you may later decide to store it as an XML file or a VCF file, or even a full fledged



database. For this, all you would need to do is add the requisite functions to handle those formats to your model. Since the rest of the application works off the model, it doesn't matter how the model is getting the data, as long as it's getting it. So you could even add functionality to store the data online, and all you would need to change is the model.

It merely provides a standard fixed way of accessing its data, and modifying it.

The View

The view is part of the application that displays data. Buttons, lists, combo boxes, etc. make up the view. It retrieves data from the model and displays it to the user; however, it does not make any changes to the data itself. Therefore, in a way, it has read-only access to the data.

An application need not have only one view. On the other hand, having multiple views easily is one of the strong points of using MVC architecture. Taking our previous example of an address book application, we could create as many views for it as we want. To add or edit contacts, you could provide multiple